Mammalia-Macaque-Dominance Network

Laura Silvana Alvarez Luque - Daniel Losada Molina

The aim of this project is to do a complete analysis of a network of interest. This includes: the network description, modeling through different techniques and results that can be obtained.

The selected network contains the dominance relations between *Macaca fuscata* females that were determined based on approach-retreat episodes around the food. The dominance range order was arranged based on these dyadic relations. Dominance is defined as "Physical contact unique to aggressive dominance interactions such as biting, head butting, fighting"

- **Vertices:** 62 Macaques.
- **Edges:** 1.167 edges representing the pairwise dominance between macaques.
- **Weights:** Frequency of the dominance.
- **Notes:** This is a animal interation network, undirected and weighted.

In [1]:
```
import zipfile
import base64
import random
import os

import numpy as np
import matplotlib.pyplot as plt
import igraph as ig
import plotly.graph_objects as go
import networkx as nx
import matplotlib.cm as cm
import matplotlib.colors as colors

from PIL import Image
from IPython.display import display
from collections import Counter

from igraph import *
```

Functions

In [2]:

```python
def unzip_file(zip_path, extract_to):
    """
    Unzip a file from zip_path into the extract_to directory.
    """
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        zip_ref.extractall(extract_to)

def get_node_info(g, node_id, threshold_pt=None):
    # Convert original ID to internal index
    internal_idx = g.vs.find(id=node_id).index

    if threshold_pt is not None:
        threshold = np.percentile(g.es["weight"], threshold_pt)
        print(f"Threshold for edge weight: {threshold}")

    incident_edges = g.incident(internal_idx, mode="ALL")
    count = 0
    node_strength = 0

    for eid in incident_edges:
        if threshold_pt is not None and g.es[eid]["weight"] < threshold:
            continue

        edge = g.es[eid]
        source = edge.source
        target = edge.target
        weight = edge["weight"]
        node_strength += weight

        source_id = g.vs[source]["id"]
        target_id = g.vs[target]["id"]
        print(f"Edge from {source_id} to {target_id}, weight = {weight}")
        count += 1

    print("Total incident edges (above threshold):", count)
    print(f"Total strength of node {node_id}:", node_strength)

def get_layout(g, layout_type="fr", seed=None):
    if seed is not None:
            np.random.seed(seed)
            np.random.seed(seed)
    if layout_type == "fr":
        print("Using Fruchterman-Reingold layout")
        return g.layout_fruchterman_reingold()
    elif layout_type == "kk":
        print("Using Kamada-Kaway layout")
        return g.layout_kamada_kawai()
    elif layout_type == "davidson_harel":
        print("Using Davidson-Harel layout")
        return g.layout_davidson_harel()
    elif layout_type == "graphopt":
        print("Using Graphopt layout")
        return g.layout_graphopt()
    else:
        return g.layout(layout_type)

def plotly_graph(g):
    # Convert to NetworkX
    G_nx = g.to_networkx()
```

```
# Get layout positions
pos = nx.spring_layout(G_nx)

# Prepare edge lines
edge_x = []
edge_y = []
for u, v in G_nx.edges():
    x0, y0 = pos[u]
    x1, y1 = pos[v]
    edge_x += [x0, x1, None]
    edge_y += [y0, y1, None]

edge_trace = go.Scatter(
    x=edge_x,
    y=edge_y,
    line=dict(width=0.5, color='#888'),
    hoverinfo='none',
    mode='lines',
    showlegend=False
)

# Create "hover points" at edge midpoints
edge_hover_x = []
edge_hover_y = []
edge_text = []

for u, v, data in G_nx.edges(data=True):
    x0, y0 = pos[u]
    x1, y1 = pos[v]
    edge_hover_x.append((x0 + x1) / 2)
    edge_hover_y.append((y0 + y1) / 2)
    weight = data.get("weight", 1)
    edge_text.append(f"{u} — {v}<br>weight: {weight}")

edge_hover_trace = go.Scatter(
    x=edge_hover_x,
    y=edge_hover_y,
    mode='markers',
    hoverinfo='text',
    text=edge_text,
    marker=dict(size=5, color='rgba(0,0,0,0)'),  # invisible
    showlegend=False
)

# Prepare node positions
node_x = []
node_y = []
node_text = []
for node in G_nx.nodes():
    x, y = pos[node]
    node_x.append(x)
    node_y.append(y)
    node_text.append(f"Node {node}")

node_trace = go.Scatter(
    x=node_x,
    y=node_y,
    mode='markers+text',
    text=[str(n) for n in G_nx.nodes()],
    textposition='top center',
    hoverinfo='text',
    marker=dict(
        size=10,
```

```python
            color='lightblue',
            line_width=1
        )
    )

    # Combine everything
    fig = go.Figure(
        data=[edge_trace, edge_hover_trace, node_trace],
        layout=go.Layout(
            title='Dominance Network of Macaques',
            showlegend=False,
            hovermode='closest',
            margin=dict(b=20, l=5, r=5, t=40),
            xaxis=dict(showgrid=False, zeroline=False),
            yaxis=dict(showgrid=False, zeroline=False)
        )
    )

    fig.show()

def get_weights_of_path(g, path):
    """
    Given a path, return the weights of the edges along that path.
    """
    original_weights = []
    inv_weights = []

    for i in range(len(path) - 1):
        source = path[i]
        target = path[i + 1]

        # Get edge ID between consecutive vertices
        edge_id = g.get_eid(source, target)

        # Extract weights
        w = g.es[edge_id]["weight"]
        iw = g.es[edge_id]["inv_weight"]

        original_weights.append(w)
        inv_weights.append(iw)

    # Print results
    print("Original weights on path:", original_weights)
    print("Inverted weights on path:", inv_weights)


def plot_degree_distribution(g, title="Degree Distribution", weighted=False):
    """
    Computes degrees (weighted or unweighted), plots their distribution,
    and returns degree info including max degree and corresponding vertex/vertices.
    """
    # Choose between weighted and unweighted degree
    if weighted:
        degrees = g.strength(weights=g.es["weight"])
    else:
        degrees = g.degree()

    # Compute degree distribution
    unique_degrees, counts = np.unique(degrees, return_counts=True)
    probabilities = counts / counts.sum()

    # Plot the distribution
    plt.figure(figsize=(8, 5))
    plt.scatter(unique_degrees, probabilities, color='black')  # Dots
```

```
    plt.vlines(unique_degrees, 0, probabilities, color='blue', alpha=0.6)  # Lines

    # Labels and title
    plt.xlabel("Degree")
    plt.ylabel("Frequency")
    plt.title(title)

    # Show plot
    plt.show()

    # Get max degree and vertex/vertices
    max_degree = max(degrees)
    #vertices_with_max = [v.index for v, d in zip(g.vs, degrees) if d == max_degree
    vertices_with_max = [v['id'] for v, d in zip(g.vs, degrees) if d == max_degree]

    # Return degrees and distribution if needed later
    return degrees, unique_degrees, probabilities, max_degree, vertices_with_max

# Network Visualization Funcitons

def plot_simple_graph(g, layout_type="fr", target=None, display_img=True):

    if target is not None and os.path.exists(target):
        print(f"[✓] Skipping {target} (already exists)")
        if display_img:
            img = Image.open(target)
            display(img)
        return

    # Layout for node positions
    layout = get_layout(g,layout_type, 253)
    vertex_label=[str(i) for i in g.vs["id"]]

    # Plot using igraph's built-in plot function
    plot_obj = ig.plot(
        g,
        layout=layout,
        vertex_label=vertex_label,  # show node indices
        edge_width=[w for w in g.es["weight"]],
        bbox=(800, 800),
        margin=50,
        target=target
    )

    if display_img:
        display(plot_obj)


def plot_weighted_graph(g, layout_type="fr", target=None, display_img=True):

    if target is not None and os.path.exists(target):
        print(f"[✓] Skipping {target} (already exists)")
        if display_img:
            img = Image.open(target)
            display(img)
        return
    g_copy = g.copy()
    # Layout for node positions
    layout = get_layout(g_copy,layout_type, 253)

    weights = np.array(g_copy.es["weight"])
    threshold = np.percentile(weights, 70)
    norm = colors.Normalize(vmin=weights.min(), vmax=weights.max())
```

```python
    # Compute node strength from strong edges
    node_strength = np.zeros(g_copy.vcount())
    for e in g_copy.es:
        if e["weight"] >= threshold:
            node_strength[e.source] += e["weight"]
            node_strength[e.target] += e["weight"]

    # Normalize node strength
    min_size, max_size = 20, 80  # Adjust for igraph scale
    normalized_strength = (node_strength - node_strength.min()) / (node_strength.ma
    node_sizes = min_size + (max_size - min_size) * normalized_strength
    node_colors = [colors.to_hex(cmap(val)) for val in normalized_strength]  # Conv

    # Set vertex attributes
    g_copy.vs["size"] = node_sizes
    g_copy.vs["color"] = node_colors
    g_copy.vs["label"] = g_copy.vs["name"]
    g_copy.vs["label_size"] = 10

    # Set edge attributes
    for e in g_copy.es:
        w = e["weight"]
        if w >= threshold:
            e["color"] = colors.to_hex(cmap(norm(w)))
            e["width"] = 1.5
            e["alpha"] = 0.6
        else:
            e["color"] = "#D3D3D3"  # Light gray for weak edges
            e["width"] = 0.3
            e["alpha"] = 0.2

    # Plot using igraph's built-in plot function
    plot_obj = ig.plot(
        g_copy,
        layout=layout,
        edge_color=g_copy.es["color"],
        edge_width=g_copy.es["width"],
        vertex_color=g_copy.vs["color"],
        vertex_size=g_copy.vs["size"],
        vertex_label=g_copy.vs["label"],
        vertex_label_size=g_copy.vs["label_size"],
        bbox=(800, 800),
        margin=50,
        target=target
    )

    if display_img:
        display(plot_obj)
```

In [3]:

```python
%%capture
def plotly_graph_with_same_image(g, image_path='./imgs/macaco_agua_sin_fondo.png'):
    # Load and encode image as base64
    with open(image_path, "rb") as image_file:
        encoded_image = base64.b64encode(image_file.read()).decode()
        image_uri = f'data:image/png;base64,{encoded_image}'

    # Convert to NetworkX
    G_nx = g.to_networkx()
    pos = nx.spring_layout(G_nx)

    # Prepare edge lines
    edge_x = []
    edge_y = []
    for u, v in G_nx.edges():
```

```
        x0, y0 = pos[u]
        x1, y1 = pos[v]
        edge_x += [x0, x1, None]
        edge_y += [y0, y1, None]

    edge_trace = go.Scatter(
        x=edge_x, y=edge_y,
        line=dict(width=0.5, color='#888'),
        hoverinfo='none', mode='lines',
        showlegend=False
    )

    # Prepare node positions and hover info
    node_x = []
    node_y = []
    node_text = []
    for node in G_nx.nodes():
        x, y = pos[node]
        node_x.append(x)
        node_y.append(y)
        node_text.append(f"Node {node}")

    node_trace = go.Scatter(
        x=node_x, y=node_y,
        mode='markers',
        hoverinfo='text',
        text=node_text,
        marker=dict(size=20, color='rgba(0,0,0,0)'),  # invisible
        showlegend=False
    )

    label_trace = go.Scatter(
        x=node_x,
        y=[y - 0.03 for y in node_y],
        mode='text',
        text=[str(n) for n in G_nx.nodes()],
        textposition='bottom center',
        hoverinfo='none',
        showlegend=False
    )

    # Add the same image to all nodes
    image_traces = []
    for x, y in pos.values():
        image_traces.append(
            dict(
                source=image_uri,
                xref="x", yref="y",
                x=x - 0.05, y=y + 0.05,
                sizex=0.1, sizey=0.1,
                xanchor="left", yanchor="top",
                sizing="stretch",
                opacity=1,
                layer="above"
            )
        )

    # Build the figure
    fig = go.Figure(
        data=[edge_trace, node_trace, label_trace],
        layout=go.Layout(
            title='Graph with Macaque Image on Each Node',
            images=image_traces,
            showlegend=False,
```

```
            hovermode='closest',
            margin=dict(b=20, l=5, r=5, t=40),
            xaxis=dict(showgrid=False, zeroline=False),
            yaxis=dict(showgrid=False, zeroline=False)
        )
    )

    fig.show()
```

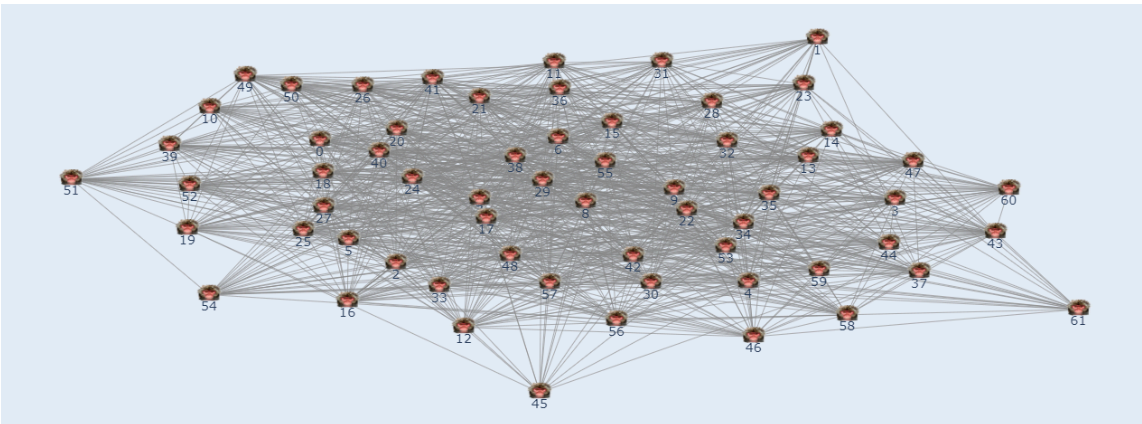
1. Input data

In [4]:
```
unzip_file('./mammalia-macaque-dominance.zip', './data')
```

In [5]:
```
#%%capture
g = ig.Graph.Read_Ncol("data/mammalia-macaque-dominance.edges", weights=True, direc
g.vs["id"] = [int(name) for name in g.vs["name"]]
#g.vs.find(id=4).index
print(g.summary())
```

```
IGRAPH UNW- 62 1167 --
+ attr: id (v), name (v), weight (e)
```

In [6]:
```
%%capture
#plotly_graph(g)
```



In [7]:
```
order = g.vcount()
size = g.ecount()

print(f"The order of the network is: {order} (vertices)")
print(f"The size of the network is: {size} (edges)")
```

```
The order of the network is: 62 (vertices)
The size of the network is: 1167 (edges)
```

We are dealing with a highly connected graph, as we can see by the edge-vertex ratio:

$$\frac{|E|}{|V|} = \frac{1167}{62} \approx 18.82$$

This means that in average, we have 18.82 edges per vertex. That's why is not surprising to find that we only have one big component that contains all vertices. Also for this reason, we don't have any subnetwork of interest and all the analysis will be done in the whole network.

In [8]:
```
components = g.components()
print(f"Number of subnetworks: {len(components)}")
```

```
Number of subnetworks: 1
```

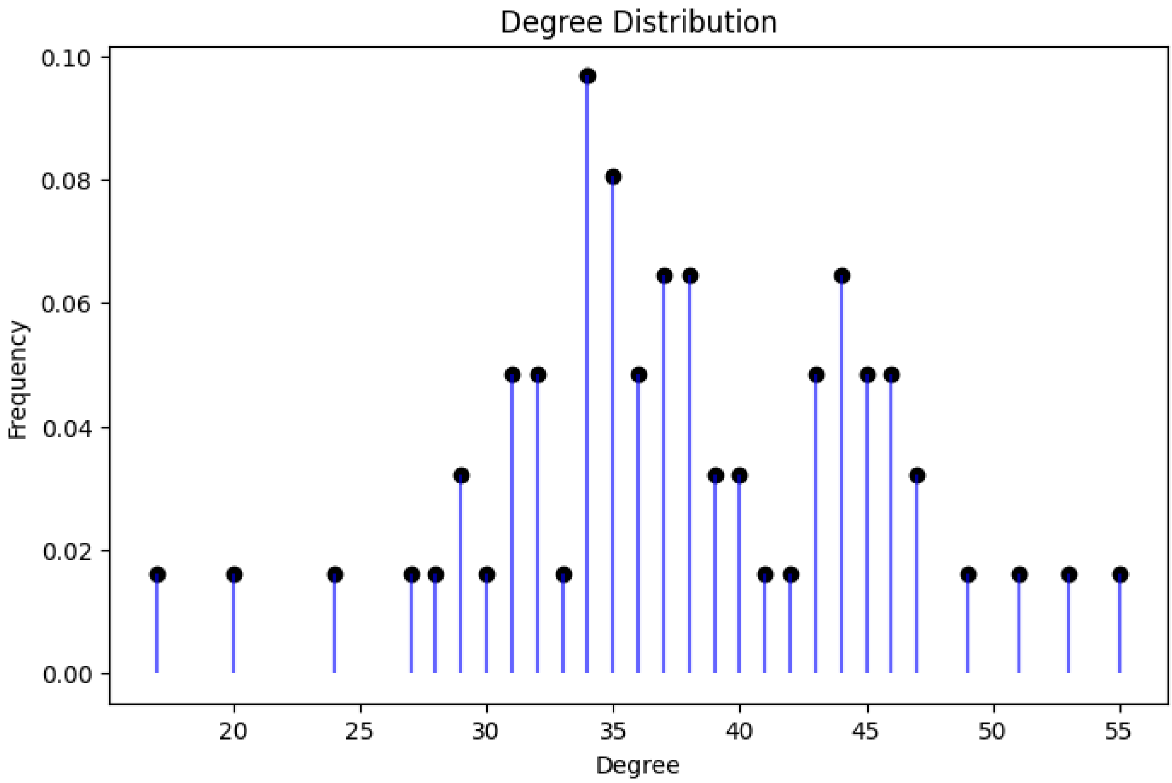
Due to the nature of our data, loops make no sense, as the edges represent interaction between two individuals. Anyways we check that our data has zero edges with same source and target.

In [9]:
```
sum(g.is_loop())
```

Out[9]:
```
0
```

With the degree distribution we can check that the network is highly connected as we saw previously. It does not have the clasical decreasing distribution starting in small degree values, instead, it starts in 14 and seems to have a bimodal distribution with concentrations around 34 and 44. Also, the most conected vertex is the 29 with 55 edges connected to it.

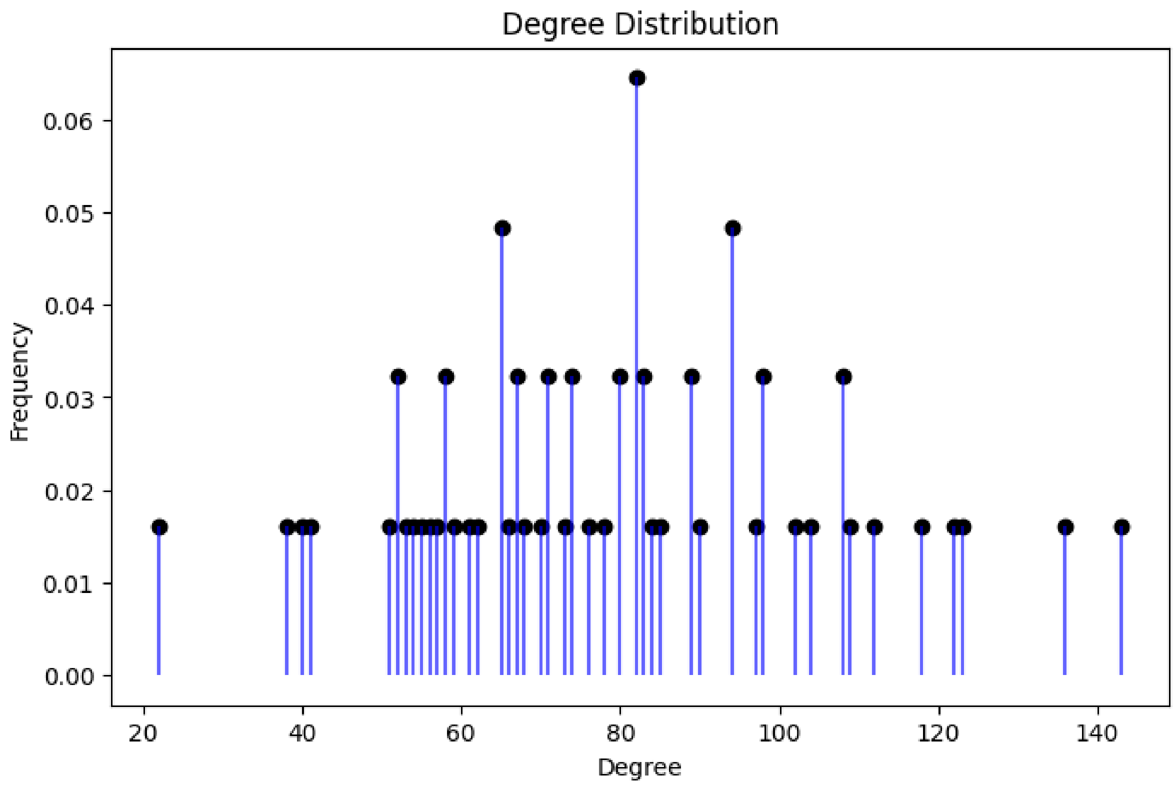
In [10]:
```
degrees, unique_deg, probs, max_deg, max_vertices = plot_degree_distribution(g)

print(f"Max degree: {max_deg}")
print(f"Vertices with max degree: {max_vertices}")
```




```
Max degree: 55
Vertices with max degree: [48]
```

Analysing the weighted degree distribution, the bimodal distribution is lost, and the maximum is centered around 80. On the other hand, the minimum value remains large, and the most connected vertex is still the 29.

In [11]:
```
degrees_w, unique_deg_w, probs_w, max_deg_w, max_vertices_w = plot_degree_distribut

print(f"Max degree: {max_deg_w}")
print(f"Vertices with max degree: {max_vertices_w}")
```




```
Max degree: 143.0
Vertices with max degree: [48]
```

Next we are going to compute the diameter.

In [12]:
```
# Diameter (unweighted)
diameter = g.diameter(directed=False, weights=None)
print("Unweighted diameter:", diameter)

# Get the actual pair of farthest nodes and the path
farthest_pair_idx = g.get_diameter(directed=False, weights=None)
farthest_path_names = [g.vs[i]["id"] for i in farthest_pair_idx]

print("Farthest vertices (unweighted) igraph idx:", farthest_pair_idx)
print("Farthest vertices (unweighted original IDs):", farthest_path_names)
```

```
Unweighted diameter: 2
Farthest vertices (unweighted) igraph idx: [0, 1, 39]
Farthest vertices (unweighted original IDs): [1, 2, 29]
```

As we already seen, our graph is highly connected, so that the unweighted diameter is 2 is quite intuitive. That means that between two indivituals, there is a path of at most 2 (another individual in the middle). But it's not a special case to have a shortest distance of 2, in fact we have 1448 paths of length 2. This means that there are 1448 pairs of individuals that are not directly connected, but they are connected through another individual.

In [13]:
```
sp_lengths = g.distances(weights=None)  # or .shortest_paths()

distances = [dist for row in sp_lengths for dist in row if dist != 0]
dist_count = Counter(distances)
print(dist_count)
```

```
Counter({1: 2334, 2: 1448})
```

To compute the weighted diameter, we need to invert the weights, as in our case higher weight means higher connection.

In [14]:
```
# Add inverse weights (handle division by zero if needed)
g.es["inv_weight"] = [1/w if w != 0 else float("inf") for w in g.es["weight"]]

# Then compute diameter using this new attribute
diameter_w = g.diameter(directed=False, weights="inv_weight")
path_w_idx = g.get_diameter(directed=False, weights="inv_weight")
path_w_names = [g.vs[i]["id"] for i in path_w_idx]

print("Weighted diameter (based on interaction):", diameter_w)
print("Farthest vertices (based on interaction) igraph idx:", path_w_idx)
print("Farthest path (based on interaction original IDs):", path_w_names)
```

```
Weighted diameter (based on interaction): 1.0833333333333333
Farthest vertices (based on interaction) igraph idx: [45, 43, 56, 54]
Farthest path (based on interaction original IDs): [6, 61, 60, 46]
```

The longest shortest path is connected through a path whose total inverse-weight is 1.0833. Notice that using the weights now we obtain a farthest path that contains 3 edges that connect 4 individuals.

In [15]:
```
get_weights_of_path(g, path_w_idx)
```

```
Original weights on path: [3.0, 2.0, 4.0]
Inverted weights on path: [0.3333333333333333, 0.5, 0.25]
```

Finally, we compute the adjancency matrix and compare the results previously obtained.

In [16]:
```
# Get the adjacency matrix
adj_matrix = np.array(g.get_adjacency().data)
adj_matrix_w = np.array(g.get_adjacency(attribute="weight").data)

# Print the adjacency matrix
print(adj_matrix)
print(adj_matrix_w)
```

```
[[0 1 1 ... 0 0 0]
 [1 0 0 ... 0 0 0]
 [1 0 0 ... 0 0 0]
 ...
 [0 0 0 ... 0 1 0]
 [0 0 0 ... 1 0 0]
 [0 0 0 ... 0 0 0]]
[[0. 3. 1. ... 0. 0. 0.]
 [3. 0. 0. ... 0. 0. 0.]
 [1. 0. 0. ... 0. 0. 0.]
 ...
 [0. 0. 0. ... 0. 4. 0.]
 [0. 0. 0. ... 4. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]]
```

In [17]:
```
# Results from the adjacence matrix

## degrees
degrees_adj = adj_matrix.sum(axis=0)

## directed or undirected
is_symmetric = np.allclose(adj_matrix, adj_matrix.T)

## Loops
self_loops = np.trace(adj_matrix)

## Nodes with degree 0 (isolated)
isolated = np.where(adj_matrix.sum(axis=1) == 0)[0]
```

```
print(degrees_adj)
print(is_symmetric)
print(self_loops)
print(isolated)
```

```
[38 24 44 36 38 41 49 46 46 47 31 38 34 35 36 42 35 53 40 35 45 39 35 32
 47 40 35 44 33 55 43 32 39 31 45 43 37 32 51 34 46 37 45 37 34 17 29 38
 44 34 31 30 34 43 27 36 37 44 29 34 28 20]
True
0
[]
```

The density is defined as:

$$Density = \frac{2 \times edges}{n(n-1)}$$

So is making a relation between the connections and the number of vertices. A high number means a dense network, while a small number a sparse one.

In [18]:

```
# Compute and print density
density = g.density(loops=False)
print(f"\nNetwork density: {density:.4f}")

# Interpret
if density > 0.5:
    print("This is a dense network.")
elif density < 0.1:
    print("This is a sparse network.")
else:
    print("This is a moderately connected network.")
```

```
Network density: 0.6171
This is a dense network.
```

Network Visualization

Energy Function

Davidson-Harel layout

For displaying the networks, the next color gradient is going to be used to represent the weigths. Also, more important vertex are displayed bigger and the edges with weights less than 2, are displayed in light gray in order to avoid a more crowded plot.

In [19]:

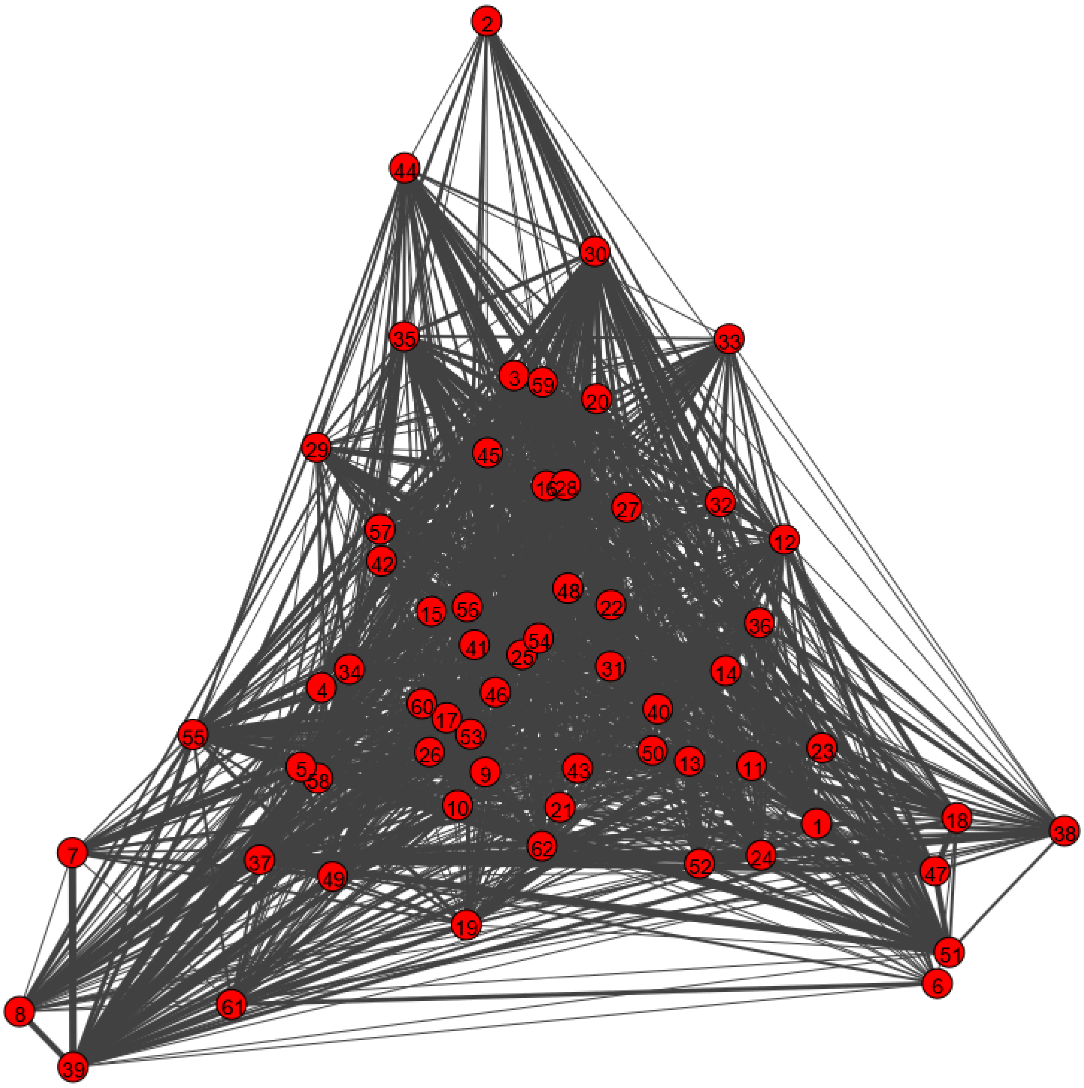
```
cmap = cm.get_cmap('viridis', 10)
cmap
```

```
C:\Users\danie\AppData\Local\Temp\ipykernel_33008\3613669891.py:1: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead.
  cmap = cm.get_cmap('viridis', 10)
```

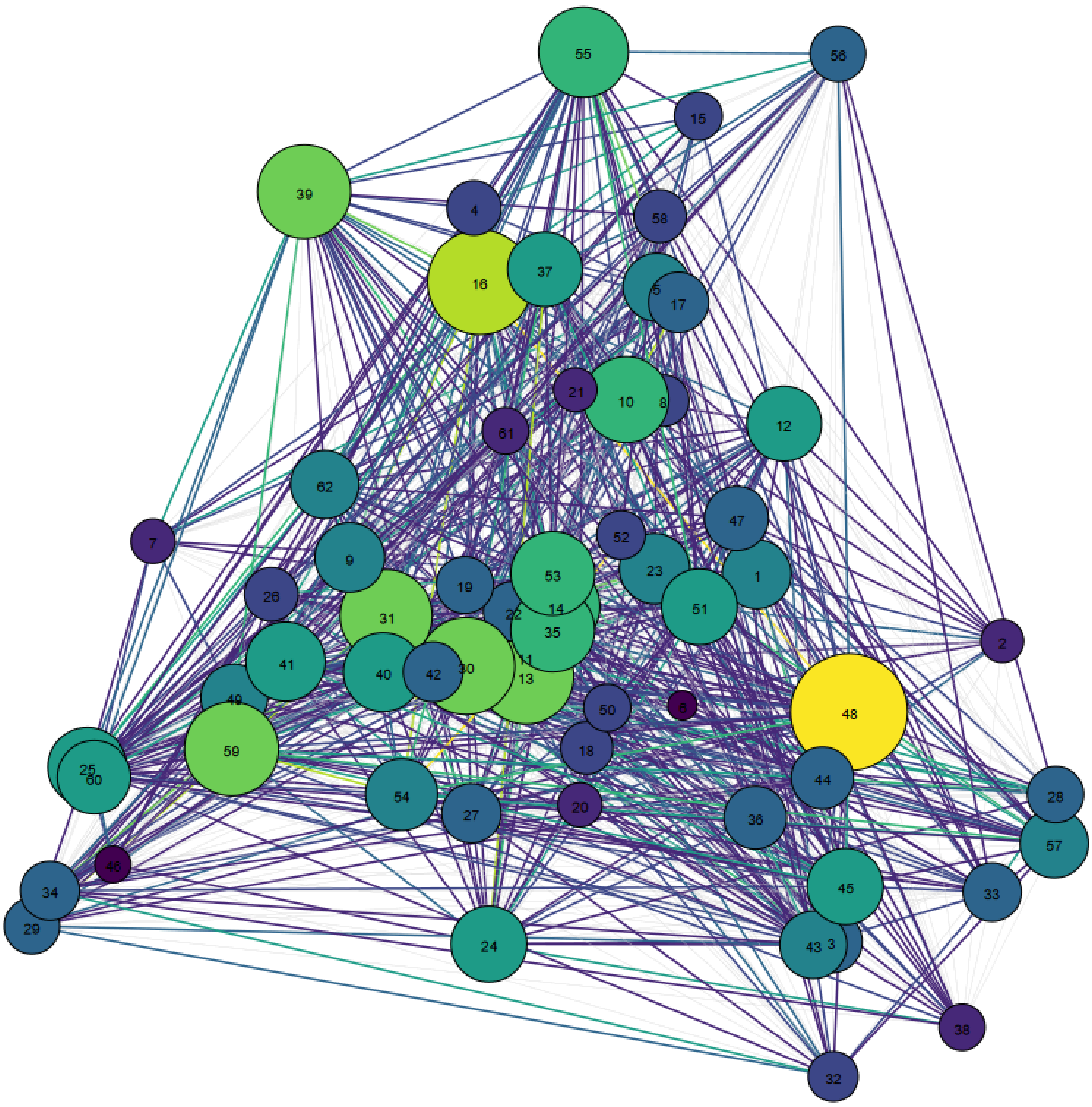
Out[19]: **viridis**

In [20]:
```python
filename = "./imgs/layout_davidson_harel_simple.png"
plot_simple_graph(g, layout_type="davidson_harel", target=filename, display_img=Tru

filename = "./imgs/layout_davidson_harel.png"
plot_weighted_graph(g, layout_type="davidson_harel", target=filename, display_img=T
```

```
[✓] Skipping ./imgs/layout_davidson_harel_simple.png (already exists)
```



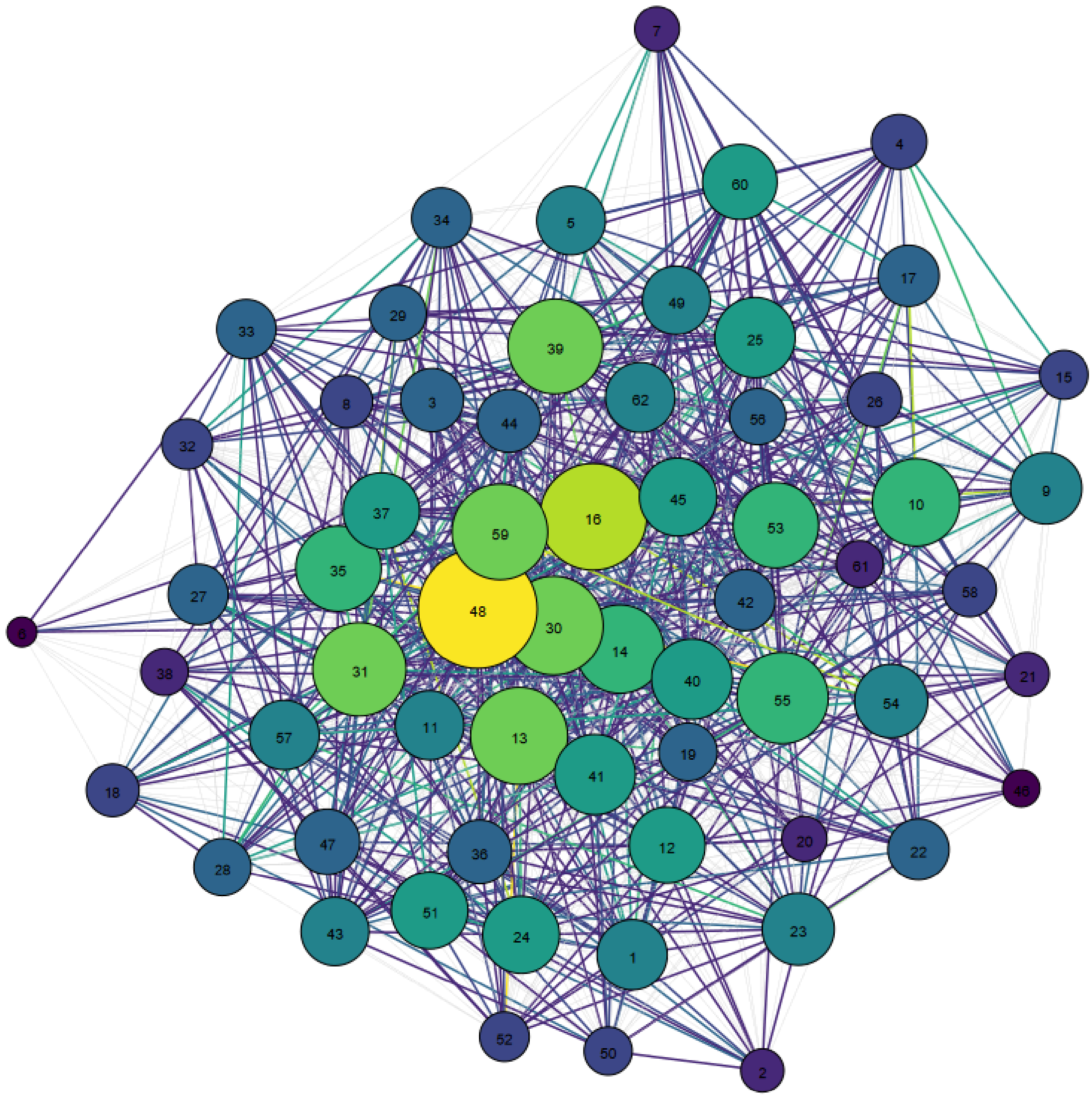
```
[✓] Skipping ./imgs/layout_davidson_harel.png (already exists)
```



Fruchterman-Reingold layout

In [21]:
```
filename = "./imgs/layout_fr.png"
plot_weighted_graph(g, layout_type="fr", target=filename, display_img=True)
```

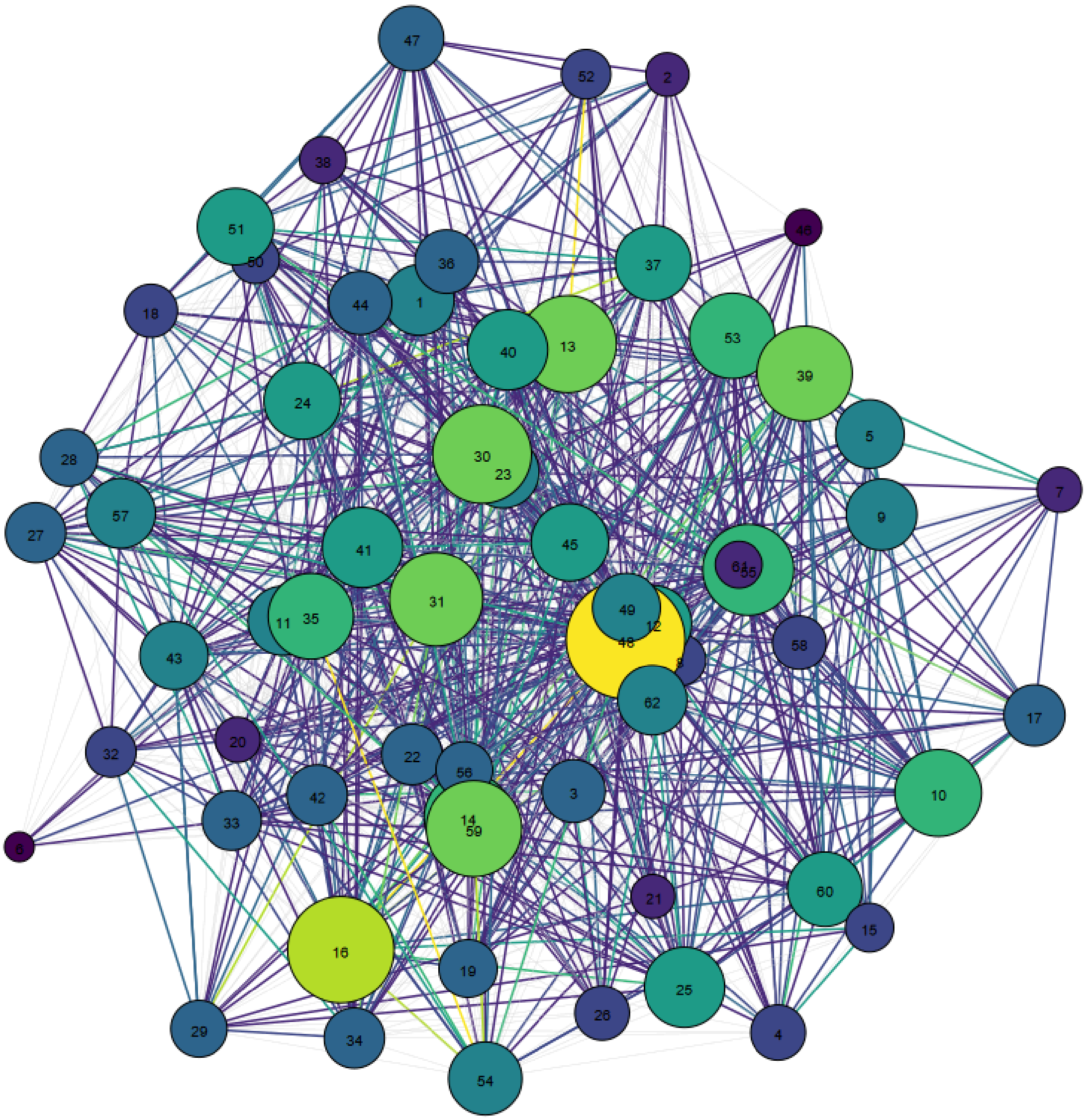
```
[✓] Skipping ./imgs/layout_fr.png (already exists)
```



Graphopt layout

In [22]:
```
filename = "./imgs/layout_graphopt.png"
plot_weighted_graph(g, layout_type="graphopt", target=filename, display_img=True)
```

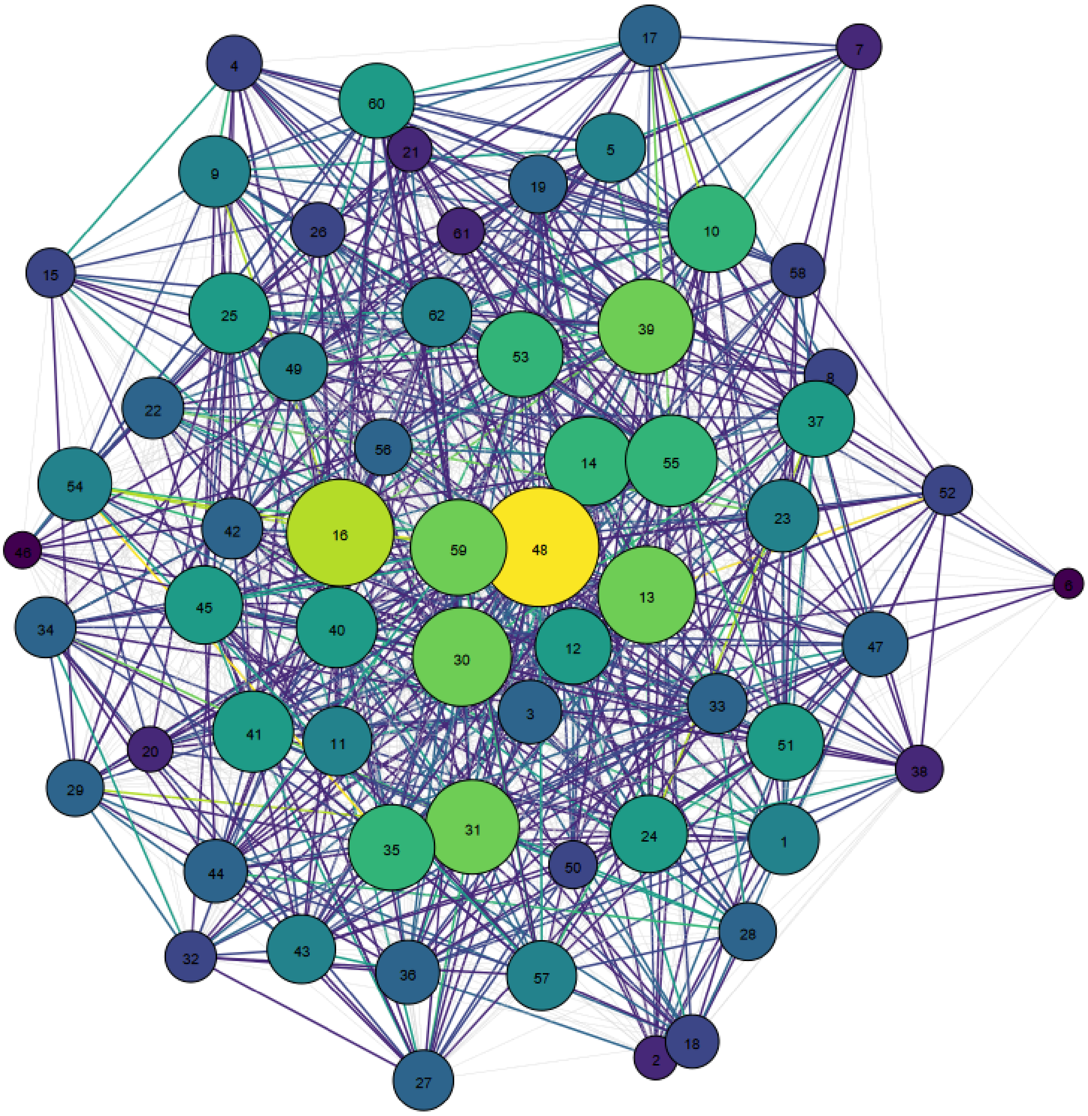
```
[✓] Skipping ./imgs/layout_graphopt.png (already exists)
```



Kamada-Kawai layout

In [23]:
```
filename = "./imgs/layout_kk.png"
plot_weighted_graph(g, layout_type="kk", target=filename, display_img=True)
```

```
[✓] Skipping ./imgs/layout_kk.png (already exists)
```



Comparison

Although we can not see much in the graph some insights can be obtained:

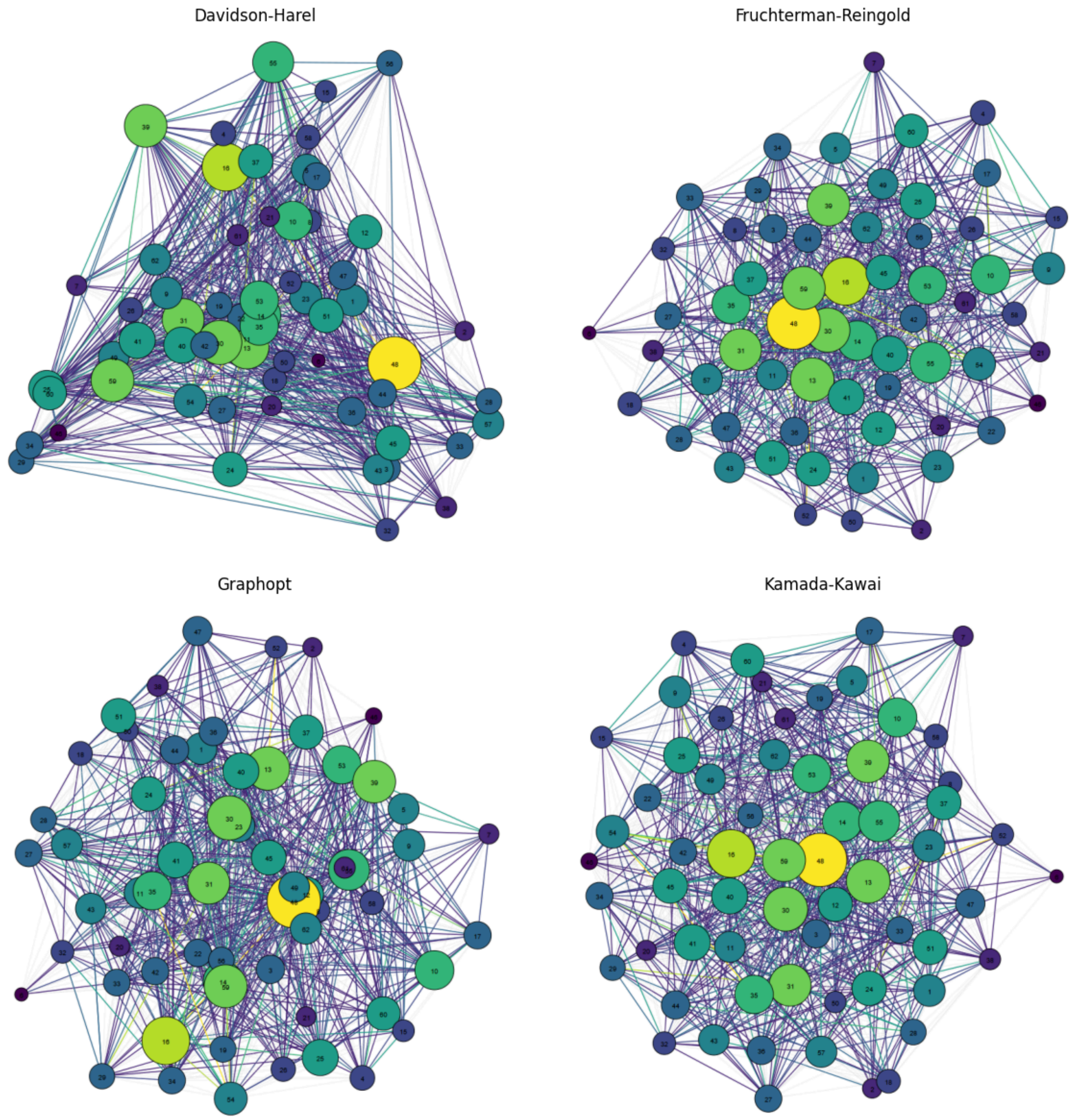
- Macaque 6 seems to have a great separation from the rest of the network in most of the layouts, except with the Davidson-Harel layout.
- Graphopt, Fruchterman-Reingold and Kamada-Kawai displays a more circular network leaving the less important vertex in the outskirts of the plot, while the other has a triangular shape separating groups of vertices, and mixing vertices of all importances in the center.
- The macaque with most importance (48) is displayed in the center with most of the layouts.

In general, all the layouts gaves similar representations of the graph, except of Davidson-Harel.

In [24]:
```
import matplotlib.pyplot as plt
from PIL import Image
```

```
# Layouts and labels
layouts = ["davidson_harel", "fr", "graphopt", "kk"]
layout_titles = ["Davidson-Harel", "Fruchterman-Reingold", "Graphopt", "Kamada-Kawa

# Save image paths
img_paths = []

# Generate and save plots using your updated function
for layout in layouts:
    filename = f"./imgs/layout_{layout}.png"
    plot_weighted_graph(g, layout_type=layout, target=filename, display_img=False)
    img_paths.append(filename)

# Display them in a 2x2 grid
fig, axs = plt.subplots(2, 2, figsize=(12, 12))

for ax, img_path, title in zip(axs.flat, img_paths, layout_titles):
    img = Image.open(img_path)
    ax.imshow(img)
    ax.set_title(title)
    ax.axis('off')

plt.tight_layout()
plt.show()
```

```
[✓] Skipping ./imgs/layout_davidson_harel.png (already exists)
[✓] Skipping ./imgs/layout_fr.png (already exists)
[✓] Skipping ./imgs/layout_graphopt.png (already exists)
[✓] Skipping ./imgs/layout_kk.png (already exists)
```



Multidimensional Scaling (MDS)

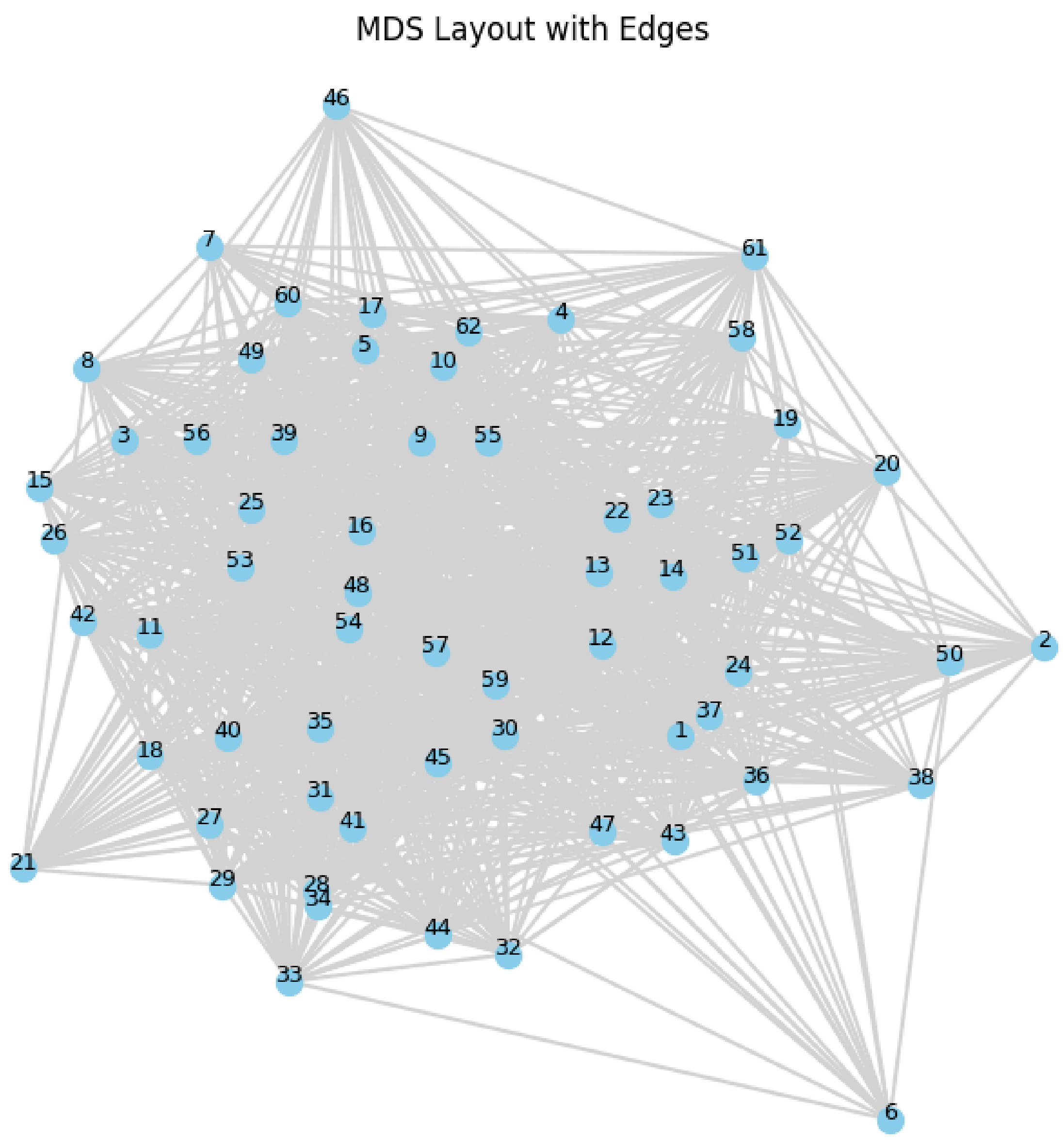
We create a 2D representation of the graph throught the MDS, where the matrix of distances are the shortest paths taking into account the inverse weights.

In [25]:
```
from sklearn.manifold import MDS
import numpy as np

# Compute the shortest path distance matrix
dist_matrix = g.shortest_paths(weights="inv_weight")

# Run MDS
mds = MDS(n_components=2, dissimilarity='precomputed', random_state=42)
mds_coords = mds.fit_transform(dist_matrix)
```

```
C:\Users\danie\AppData\Local\Temp\ipykernel_33008\2062823136.py:5: DeprecationWarn
ing: Graph.shortest_paths() is deprecated; use Graph.distances() instead
  dist_matrix = g.shortest_paths(weights="inv_weight")
```

Initially, we made a plot without displaying the weights. The result does not seems to be more informative than what we saw before and the plot is still caothic giving not a lot of information.

In [26]:
```python
plt.figure(figsize=(8, 8))

# Plot edges
for edge in g.get_edgelist():
    x0, y0 = mds_coords[edge[0]]
    x1, y1 = mds_coords[edge[1]]
    plt.plot([x0, x1], [y0, y1], color='lightgray', zorder=1)

# Plot nodes
plt.scatter(mds_coords[:, 0], mds_coords[:, 1], s=100, color='skyblue', zorder=2)

# Add labels
for i, label in enumerate(g.vs["name"]):
    plt.text(mds_coords[i, 0], mds_coords[i, 1], label, fontsize=9, ha='center', zo

plt.title("MDS Layout with Edges")
plt.axis('off')
plt.show()
```

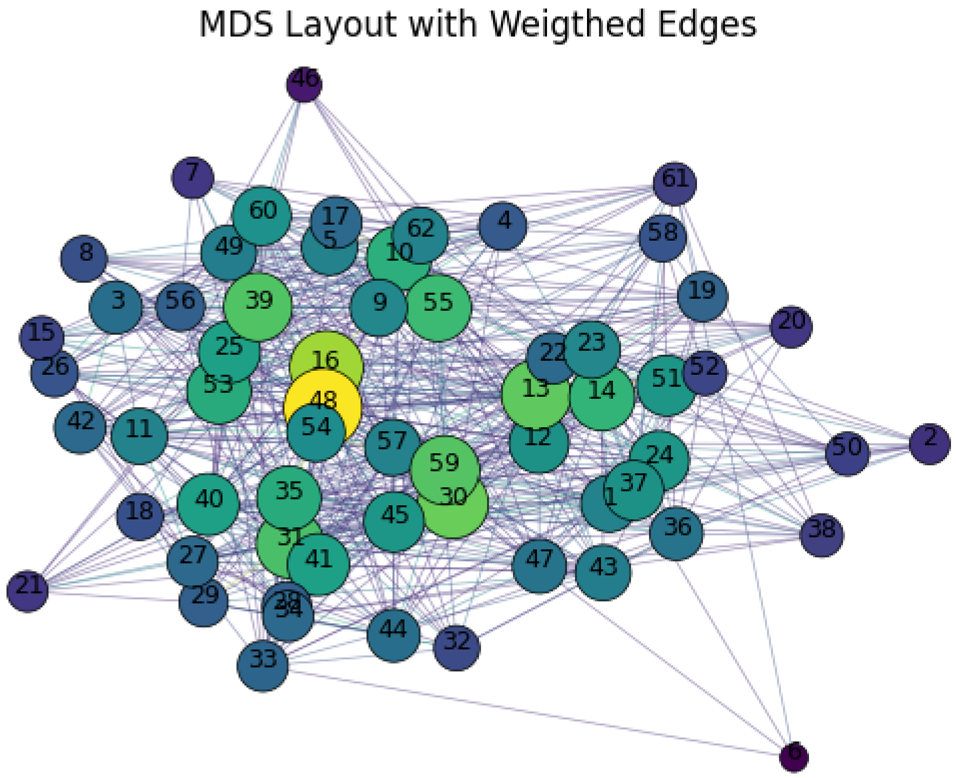


Then, we applied the color and size gradient as before, and we can see that the Multidimensional Scaling is by far the best representation that we have obtained of the network.

The most important vertex (Macaque 48) is in some way centered, and around it we can diferenciate two "rings". Closer to this node, the following most important macaques, and on the outskirts of the plot, the less important ones. Allowing to see a kind of clustering based on the dominance-relations.

In [27]:
```
weights = np.array(g.es["weight"])
inv_weights = np.array(g.es["inv_weight"])
threshold = np.percentile(weights, 60)
norm = colors.Normalize(vmin=weights.min(), vmax=weights.max())
cmap = cm.get_cmap('viridis')
print("threshold: ", threshold)
edges_to_plot = [e.index for e in g.es if e["weight"] >= threshold]

# Compute node strength from strong edges
node_strength = np.zeros(g.vcount())
for e in g.es:
    if e["weight"] >= threshold:
        node_strength[e.source] += e["weight"]
        node_strength[e.target] += e["weight"]

# Normalize node strength to size and color
min_size, max_size = 100, 800
normalized_strength = (node_strength - node_strength.min()) / (node_strength.max()
node_sizes = min_size + (max_size - min_size) * normalized_strength
node_colors = [cmap(val) for val in normalized_strength]

for e in g.es:
    if e.index not in edges_to_plot:
        continue  # Skip plotting this weak edge
    i, j = e.source, e.target
    x0, y0 = mds_coords[i]
    x1, y1 = mds_coords[j]
    w = e["weight"]
    color = cmap(norm(w))
    plt.plot([x0, x1], [y0, y1], color=color, linewidth=0.5, alpha=0.5, zorder=1)

# Plot nodes
plt.scatter(
    mds_coords[:, 0], mds_coords[:, 1],
    s=node_sizes, color=node_colors,
    edgecolor='black', linewidth=0.5, zorder=2
)

# Add labels
for i, label in enumerate(g.vs["name"]):
    plt.text(mds_coords[i, 0], mds_coords[i, 1], label, fontsize=9, ha='center', zo

plt.title("MDS Layout with Weigthed Edges")
plt.axis('off')
plt.show()
```

```
C:\Users\danie\AppData\Local\Temp\ipykernel_33008\3662256221.py:5: MatplotlibDepre
cationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be
removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_
cmap()`` or ``pyplot.get_cmap()`` instead.
  cmap = cm.get_cmap('viridis')
threshold:  2.0
```



In [28]:
```
#get_node_info(g, 6, threshold_pt=60)
```